

CAMPUS LOST & FOUND SYSTEM

Student Name	Vedansh Singh Tomar
Registration Number	25BAI11509
Branch	B.Tech CSE (Artificial Intelligence & Machine Learning)
Year	2nd Year
Course	Programming in Java
Project	Campus Lost & Found System

1. Introduction

Campus Lost & Found System is a Java-based command-line application designed to provide a centralized way for students to record and manage lost and found items on a college campus. The application allows users to report lost items, report found items, search records, submit claim requests, track returned items, view personal reports, and generate a summary report.

The project applies core Java programming concepts including classes and objects, encapsulation, constructors, methods, ArrayList collections, file handling, exception handling, conditional statements, loops, and modular programming.

2. Problem Statement

Students frequently lose or find personal and academic belongings around a college campus. Informal communication through messaging groups may make it difficult to locate the correct owner or maintain an organized record. The Campus Lost & Found System provides a simple centralized application for recording, searching, claiming, and tracking lost and found items.

3. Objectives

- Provide a structured way to report lost and found items.
- Allow users to search reported items using item name, category, or location.
- Provide a mechanism for submitting claim requests for found items.
- Track the status of items, including returned items.
- Maintain item records using local file storage.
- Generate a summary report of lost, found, and returned items.
- Demonstrate practical application of Java programming concepts.

4. Functional Requirements

FR1 – Report Lost Item: The system shall allow students to report a lost item by entering the item name, category, location, date, description, and reporter name.

FR2 – Report Found Item: The system shall allow students to report an item that they have found using relevant item details.

FR3 – Search Items: The system shall allow users to search reported items using item name, category, or location.

FR4 – View All Items: The system shall display all lost and found items currently stored in the system.

FR5 – Claim Found Item: The system shall allow a user to submit a claim request for an available found item using its Item ID.

FR6 – Mark Item as Returned: The system shall allow an item to be marked as returned after recovery.

FR7 – View My Reports: The system shall allow users to view items reported under their name.

FR8 – Student Profile: The system shall allow a user to enter and display name, student ID, and email.

FR9 – View Claim Requests: The system shall display submitted claim requests with Item ID, claimant name, and status.

FR10 – Generate Report: The system shall generate a summary containing total reports, lost items, found items, and returned items.

FR11 – Data Persistence: The system shall save item records to a local text file and load previously saved records when the application starts.

FR12 – Input Validation: The system shall prevent empty required fields and handle invalid Item IDs and menu choices.

5. Non-Functional Requirements

NFR1 – Usability: The system should provide a simple menu-driven interface for students.

NFR2 – Reliability: The system should handle invalid inputs and missing Item IDs without unexpected termination.

NFR3 – Performance: Search, reporting, and item-management operations should require minimal processing time for normal campus-sized data.

NFR4 – Maintainability: Separate Java classes should keep the implementation modular and easier to maintain.

NFR5 – Data Persistence: Reported item information should remain available after application restart through local file storage.

NFR6 – Portability: The application should run on systems with a compatible Java Development Kit installed.

NFR7 – Error Handling: Meaningful error messages should be provided for invalid menu choices, empty fields, and non-numeric Item IDs.

6. System Architecture

The application follows a simple modular architecture. Main.java acts as the application entry point and interacts with ItemManager for core operations. InputHelper handles validated input. Item represents lost/found item records, ClaimRequest represents claim requests, User represents a student profile, and ReportGenerator produces summary reports. ItemManager also performs local file persistence through data/items.txt.

Campus Lost & Found System

Architecture Diagram

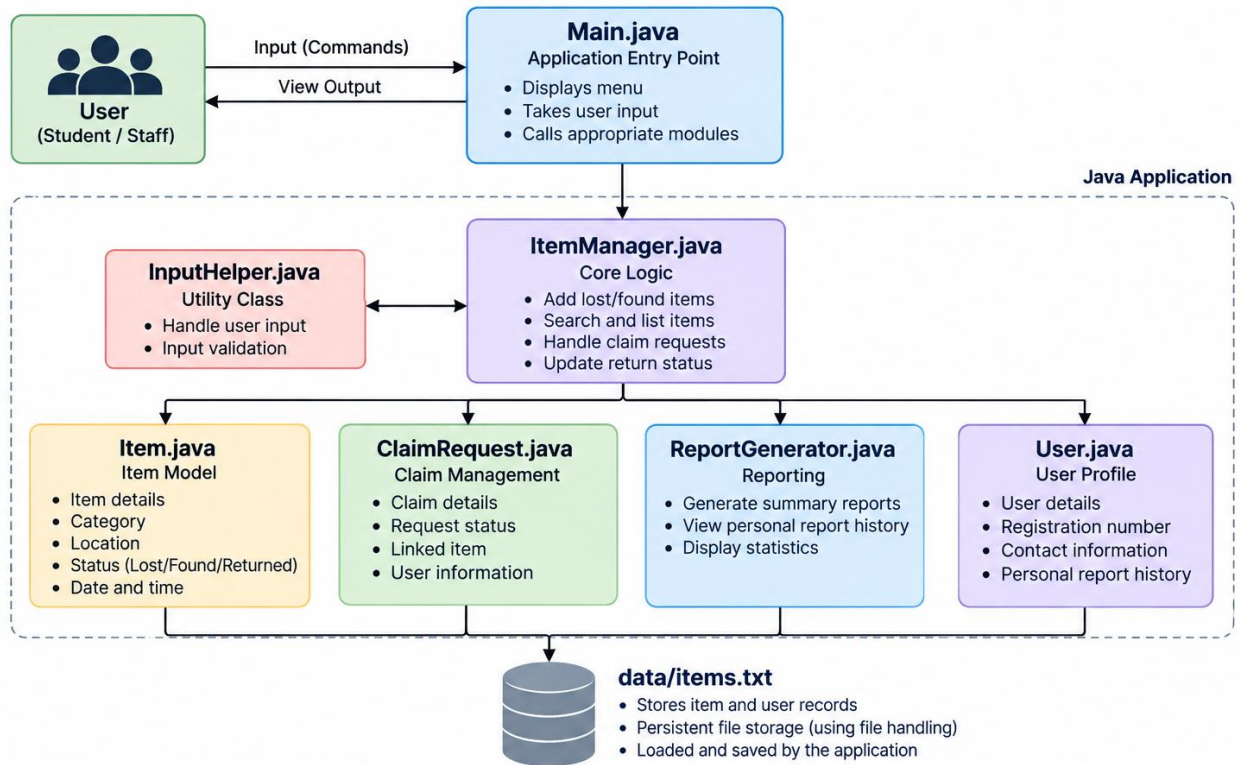


Figure 1. Architecture Diagram

7. System Design

7.1 UML Use Case Diagram

The use case diagram represents the major operations available to a Student/User, including reporting, searching, claiming, returning, profile viewing, claim-request viewing, and report generation.

Campus Lost & Found System UML Use Case Diagram

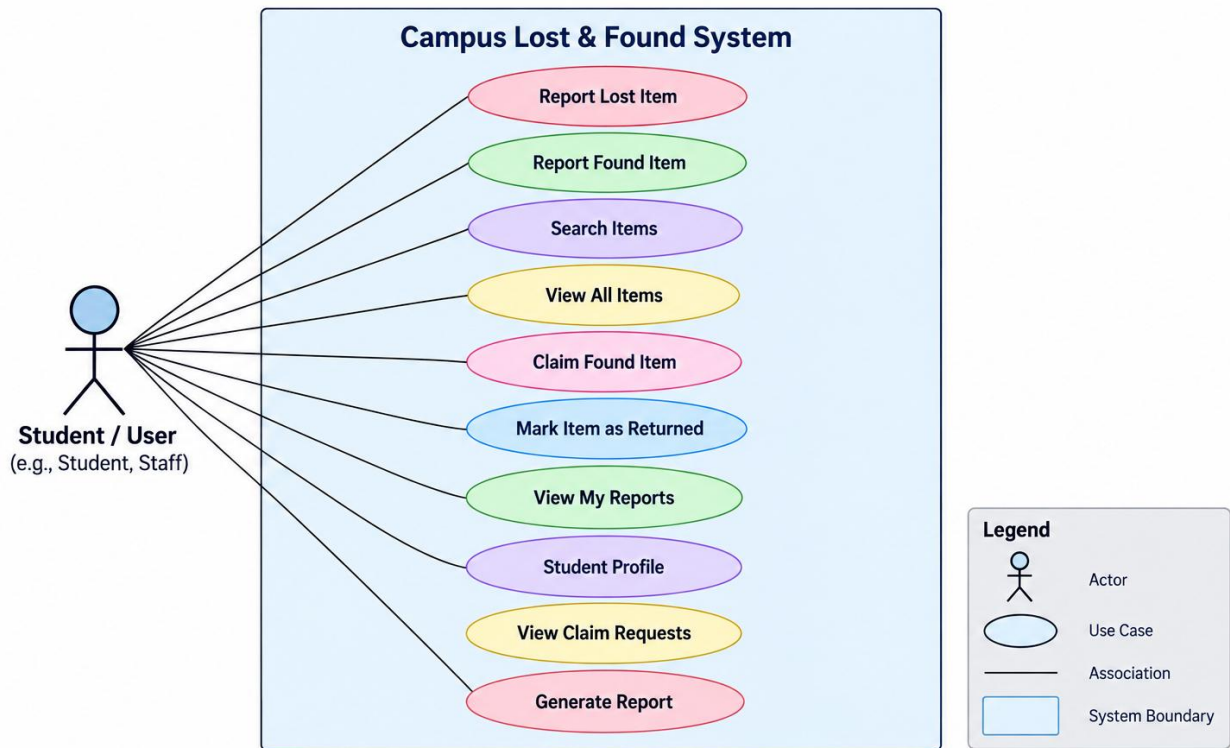


Figure 2. UML Use Case Diagram

7.2 UML Class Diagram

The class diagram represents the Java classes used in the implementation and the relationships between the main application, item management, item records, claim requests, user profile, input helper, and report generator.

Campus Lost & Found System

UML Class Diagram

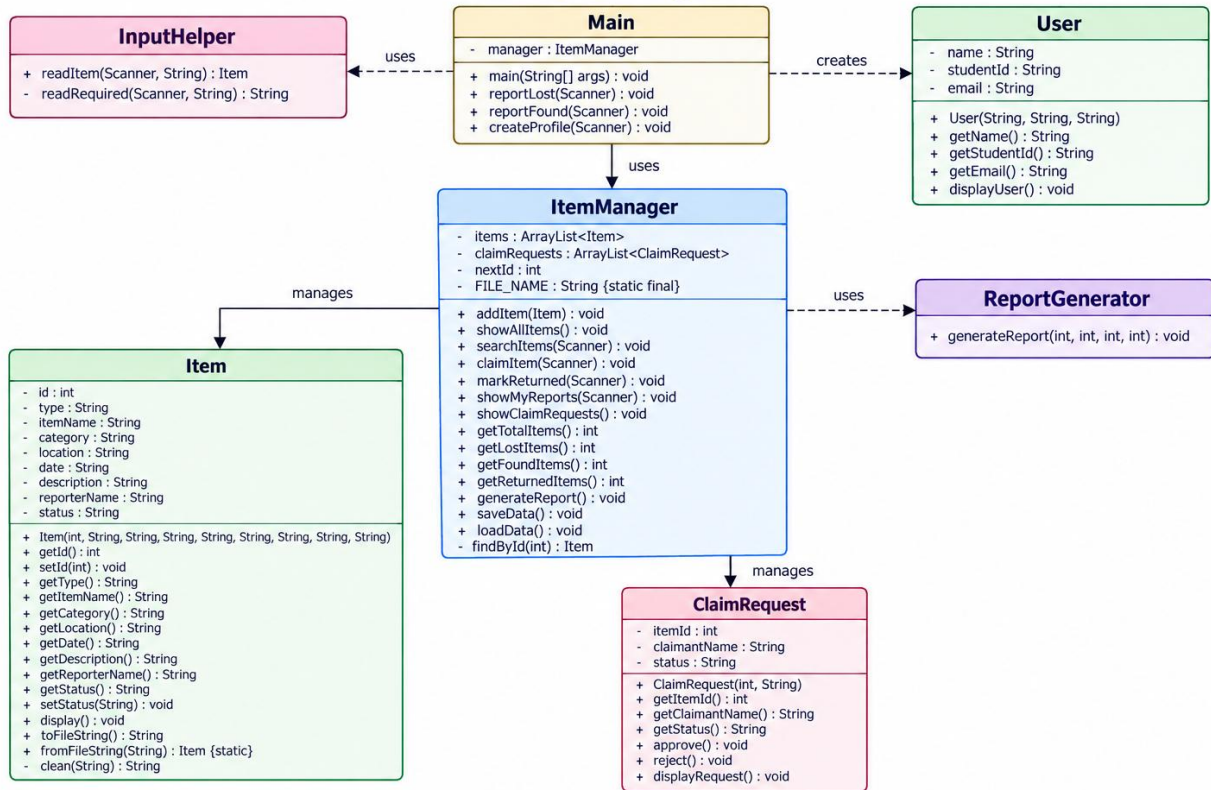


Figure 3. UML Class Diagram

7.3 UML Sequence Diagram

The sequence diagram shows the interaction flow for reporting a lost item: the user selects the operation, item details are collected and validated, an Item object is created, ItemManager assigns an ID and stores the record, and the item data is saved to the local file.

Campus Lost & Found System

UML Sequence Diagram (Report Lost Item)

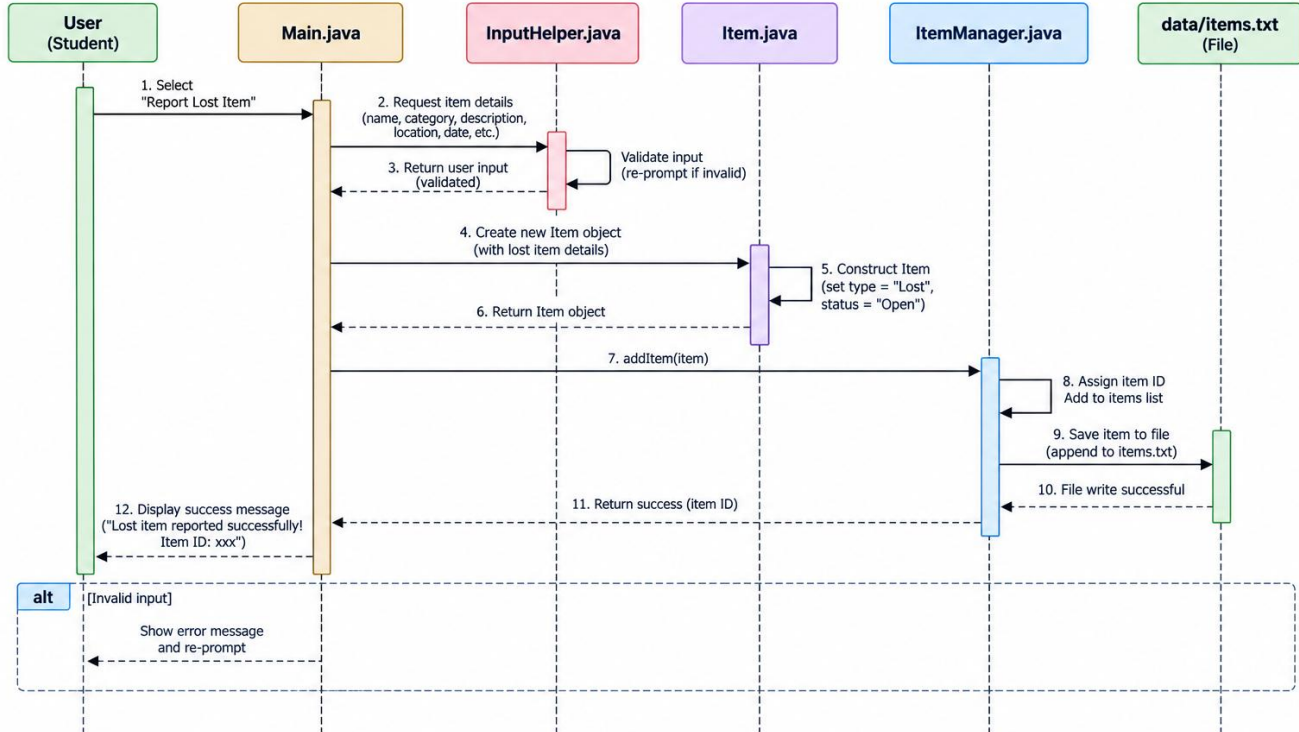


Figure 4. UML Sequence Diagram – Report Lost Item

8. Java Concepts Applied

Object-Oriented Programming: The system is divided into classes representing application components and data entities.

Encapsulation: Class fields are kept private and accessed through public getter/setter methods.

Classes and Objects: Item, User, ClaimRequest, ItemManager, InputHelper, and ReportGenerator are implemented as Java classes.

Constructors: Constructors initialize Item, User, and ClaimRequest objects.

ArrayList: ItemManager uses ArrayList to maintain item and claim-request collections.

File Handling: FileReader, BufferedReader, FileWriter, and PrintWriter are used for local item-data persistence.

Exception Handling: NumberFormatException and IOException are handled to provide safer input and file operations.

Modular Programming: Different responsibilities are separated into multiple Java source files.

Methods and Conditional Statements: Application operations are implemented using methods and decision logic.

Loops: Loops are used for menu repetition, searching, displaying records, and calculating report counts.

String Handling: String methods such as trim(), toLowerCase(), equalsIgnoreCase(), and contains() are used for input and search operations.

9. Implementation

The source code is organized into the following meaningful modules/classes:

Java File	Responsibility
Main.java	Application entry point, menu handling, lost/found reporting, and profile creation.
Item.java	Data model for lost and found item records, including serialization to and from file format.
ItemManager.java	Core business logic for adding, searching, claiming, returning, reporting, and persistence.
InputHelper.java	Input collection and validation for required fields.
User.java	Student profile representation and display.
ClaimRequest.java	Claim request representation and request status handling.
ReportGenerator.java	Generation of summary statistics for the application.

9.1 Data Storage

The current implementation uses a local text file named data/items.txt. Item records are converted to a pipe-separated format using Item.toFileString() and reconstructed using Item.fromFileString().

ItemManager.loadData() reads existing records when the program starts, while saveData() writes the current item collection to the file.

10. Screenshots and Results

The application was tested through the command-line menu. The following operations were executed during testing:

- Reported a lost item and received an automatically assigned Item ID.
- Reported a found item and displayed its available status.
- Searched for an item using a keyword.
- Tested a search with no matching record.
- Submitted a claim request for an available found item.
- Displayed claim requests and verified the request status as PENDING.
- Marked an item as returned.
- Viewed reports associated with a student's name.
- Created and displayed a student profile.
- Verified that item records could be loaded after restarting the application.

```
=====
                        CAMPUS LOST & FOUND
=====
1. Report Lost Item
2. Report Found Item
3. Search Items
4. View All Items
5. Claim Found Item
6. Mark Item as Returned
7. View My Reports
8. Student Profile
9. View Claim Requests
10. Generate Report
11. Exit
=====
Enter your choice: 
```

1. Application Menu / Main Screen

```
Lost item reported successfully.
Your Item ID is: 1001
```

2. Report Lost Item

```
===== CLAIM REQUEST =====
Item ID       : 1003
Claimant      : Avika
Status        : PENDING
=====
```

3. Search / Claim Request

```
=====
                        CAMPUS LOST & FOUND REPORT
=====
Total Reports   : 3
Lost Items      : 1
Found Items     : 2
Returned Items  : 1
=====
```

4. Generated Report

11. Testing

Testing focused on normal operations, invalid inputs, searching, claim handling, status updates, profile creation, report generation, and local persistence.

Test Case	Input / Action	Expected Result	Result
TC01	Report lost item with valid details	Item created and assigned an Item ID	Pass
TC02	Report found item with valid details	Found item stored with AVAILABLE status	Pass
TC03	Search for existing keyword	Matching records displayed	Pass
TC04	Search for non-existing keyword	No matching items message displayed	Pass
TC05	Claim available found item	Claim request created with PENDING status	Pass
TC06	Claim already unavailable item	System rejects the claim	Pass
TC07	Enter non-numeric Item ID	Validation/error message displayed	Pass
TC08	Mark item as returned	Item status changes to RETURNED	Pass
TC09	View my reports	Reports matching entered name displayed	Pass
TC10	Restart application	Previously saved item records loaded	Pass

12. Challenges Faced

- Designing the application as multiple cooperating Java classes rather than one large program.
- Handling user input safely when mixing menu operations and text fields.
- Implementing local file persistence while keeping the application simple.
- Managing item status transitions such as AVAILABLE, CLAIM REQUESTED, and RETURNED.
- Handling invalid numeric Item IDs and empty input without terminating the application.

13. Learning Outcomes

- Improved understanding of object-oriented programming in Java.
- Learned how to divide a problem into multiple classes and responsibilities.
- Applied ArrayList collections for managing dynamic records.
- Applied file handling for persistent storage.
- Practiced exception handling and input validation.
- Understood how UML diagrams can represent the architecture and behavior of a software system.

14. Future Enhancements

- Add a graphical user interface or web interface.
- Introduce a database instead of text-file storage.
- Add persistent storage for claim requests.
- Add authentication and role-based access for students and administrators.
- Add image upload for lost and found items.
- Add automatic matching between lost and found item descriptions.
- Add notification support for claim and return updates.

15. Conclusion

Campus Lost & Found System demonstrates how Java can be used to develop a modular application for a realistic campus problem. The project combines object-oriented programming, collections, file handling, exception handling, input validation, and modular design. The implemented features provide a structured workflow for reporting, searching, claiming, returning, and summarizing lost and found items.

16. References

- Oracle Java Documentation – Java language and standard library documentation.
- Java Platform Documentation – Reference material for Java classes and APIs.
- Course materials provided for Java Programming.
- VITyarthi project assignment guidelines.